# System Verification and Validation Plan for Software Engineering

Team 18, Team Magic
Allen Tong
Amos Edeaghe
Jacob Brodersen
Joshua Engel
Nehan Munir

November 3, 2026

# Revision History

| Date | Version | Notes |
|---|---|---|
| October 31, 2026 | 1.0 | Jacob Brodersen added rough draft of Section 3 |
| November 1, 2026 | 1.1 | Jacob Brodersen refined Section 3 |
| November 02, 2026 | 1.2 | Amos Edeaghe added Section 2 |
| November 3, 2026 | 1.3 | Allen Tong added Sections 4.1 and 4.3 |
| November 3, 2026 | 1.4 | Nehan Munir added Section 1 and Appendix 6 |
| November 3, 2026 | 1.5 | Joshua Engel added Section 4.2 and updated Section 4.3 |
| November 3, 2026 | 1.6 | All team members added reflection and Gen AI use disclosure |

# Contents


**1 Symbols, Abbreviations, and Acronyms** **iv**

**2 General Information** **1**
- 2.1 Summary . . . 1
- 2.2 Objectives . . . 1
- 2.3 Custom Documents . . . 2
- 2.4 Relevant Documentation . . . 2

**3 Plan** **3**
- 3.1 Verification and Validation Team . . . 3
- 3.2 SRS Verification . . . 4
- 3.3 Design Verification . . . 6
- 3.4 Verification and Validation Plan Verification . . . 7
- 3.5 Implementation Verification . . . 8
- 3.6 Automated Testing and Verification Tools . . . 9
- 3.7 Software Validation . . . 10

**4 System Tests** **11**
- 4.1 Tests for Functional Requirements . . . 11
  - 4.1.1 Round Setup . . . 11
  - 4.1.2 Tasks and Answer Checking . . . 13
  - 4.1.3 Scoring . . . 17
  - 4.1.4 Saving and Recovery . . . 18
  - 4.1.5 3D Visualization . . . 19
  - 4.1.6 Guided Learning and Tutorial . . . 20
  - 4.1.7 Accounts and Competitive Play . . . 21
  - 4.1.8 Wellbeing . . . 25
- 4.2 Tests for Nonfunctional Requirements . . . 25
  - 4.2.1 Performance Testing . . . 26
  - 4.2.2 Usability Testing . . . 27
  - 4.2.3 Maintainability Testing . . . 29
  - 4.2.4 Portability Testing . . . 30
  - 4.2.5 Reliability Testing . . . 31
  - 4.2.6 Security and Privacy Testing . . . 32
  - 4.2.7 Accessibility Testing . . . 34
- 4.3 Traceability Between Test Cases and Requirements . . . 35

**5 Unit Test Description** **39**
5.1 Unit Testing Scope . . . 39
5.2 Tests for Functional Requirements . . . 39
5.2.1 Module 1 . . . 39
5.2.2 Module 2 . . . 39
5.3 Tests for Nonfunctional Requirements . . . 40
5.3.1 Module ? . . . 40
5.4 Traceability Between Test Cases and Modules . . . 40

**6 Appendix** **41**
6.1 Symbolic Parameters . . . 41
6.2 Usability Survey Questions? . . . 41

**7 Appendix — Generative AI Use Disclosure** **44**
7.1 AI Tools Used . . . 44
7.2 How and Where Used . . . 44
7.3 Verification of AI-Generated Content . . . 45

**8 Appendix — Reflection** **46**

# List of Tables

2 Definitions and Acronyms . . . v
3 Verification and Validation Team . . . 3
4 Traceability Between Functional Requirements and System Tests 37
5 Traceability Between Nonfunctional Test Cases and Requirements . . . 38

# List of Figures

# 1 Symbols, Abbreviations, and Acronyms

| Term | Definition |
|---|---|
| SOLID principles | Principles to male code readable and maintainable |
| SRP | Single-Responsibility Principle: A class has a singular responsibility |
| OCP | Open-closed Principle: Open for extension, closed for modification |
| LSP | Liskov Substitution Principle: Subobects can use their base objects' functions |
| ISP | Integration Segregation Principle: Interfaces shouldn't be vast and aimless |
| DIP | Dependency Inversion Principle: Focus on abstractions |
| VnV | System Verification and Validation |
| MIS | Module Interface Specification: A standard and formal speecification that acts like the implementation is a black box |
| Base 8 | A number system using the digits 0 to 7 where the value for each digit in decimal can be found by multiplying by 8 to the power of n where n represents the number of positions from the right. |
| Octal | Another name for base 8. |
| Magic square | A square array of positive integers where the sum of each individual row, column, or main diagonal is the same. |
| Magic cube | The 3-dimensional equivalent of a magic square where the sums of each row, column, pillar, main diagonal, or triagonal is the same. |
| Magic constant | The fixed sum obtained when doing sums in any row, column, or main diagonal in a magic square. |
| Most perfect magic cube | A magic cube where all rows, columns, pillars, diagonals, triagonals, and broken space diagonals add up to the magic constant. |

| | |
|---|---|
| Triagonal | A line or set of cells connecting two opposite vertices through the interior of a cube and hence moves across all three axes. |
| Broken space diagonal | A diagonal that wraps around the cube's boundaries. |
| Voxel | A single 3D cell within the magic cube. |
| Slice | A 2D display providing all the voxels within a certain cross section of the magic cube. |
| Round | A single playthrough of the game consisting of 8 sequential tasks. |
| Starter | The initial randomly chosen visible voxel that acts as a starting point for the player to start solving tasks. |

Table 2: Definitions and Acronyms

This document provides a Verification and Validation plan for Team Magic's Magic-8 project. This document will lay out how testing will be done for the browser-based octal magic cube game as a black box, and automated tools to ensure that the source code is statically correct. It also currently contains the tests for the functional and non functional requirement, alongside user interiew questions. After the formalization and completion of the MIS, this will also contain descriptions of the unit tests.

# 2 General Information

## 2.1 Summary

The software being tested is called Magic 8. Magic 8 is a free-to-play, web-based 3D puzzle game built around an 8x8x8 magic cube. Users are given a starting cell and tasks to solve to guide their navigation through the magic cube, ending with them performing addition or subtraction in octal to confirm each task's completion. The game allows users to navigate the cube to select cells by moving through its faces and slices or by rotating the cube. Upon completing a task, the users are given a score based on the task's difficulty and the time taken to complete it. Magic 8's goal is to provide a fun and engaging resource to interact with base 8 and magic cubes that is accessible to those interested.

## 2.2 Objectives

The objective of this document is to check if the project exhibits sufficient usability and builds confidence in its correctness. The focus on usability is due to the project's game nature, with a goal of being both fun and engaging through its task-based system; insufficient usability would build frustration amongst users and hamper that experience. The web-based game would have users solving tasks with an asynchronous system to rotate, move through, and solve the tasks in the cube. As such, usability, user validation, and correctness are required for a satisfactory user experience and to encourage users to adopt the tool.

The project will make use of certain third-party libraries; due to resource constraints, the team has decided to work under the assumption that all

used third-party libraries and dependencies are already verified by their implementation teams.

## 2.3 Custom Documents

The custom document **User/Stakeholder/Client Interview Report** was chosen to provide the team with a deeper understanding of what a typical user would want from the game. Through a mix of structured and unstructured interviews of our stakeholders, we will gain a better view of how they envision gameplay and what they find intuitive or difficult so they can be considered and integrated in the validation of our system.

The other custom document that will be tackled is the **Usability Report**, which would provide similar information on how users experience the game but through observations of their interaction with the system. The goal is to use this document to measure the goal metrics set for fun, difficulty, and usability.

## 2.4 Relevant Documentation

To guide the testing of our system, the Verification and Validation plan will make use of the following documents:

- Software Requirements Specification (SRS) (**?**): The SRS details all the functional and non-functional requirements the project must meet. These requirements serve as the basis for the tests that are designed with the belief that by checking against and satisfying these requirements, the system will have satisfactory usability and correctness.

- Module Interface Specification (MIS) (**?**): The MIS details the usage, purpose, and specification of the interfaces for each module that makes up Magic 8; as such, it is a necessary reference for verifying the interactions between the interfaces and the system.

- Module Guide (MG) **?**: This provides the layout for the modules that make up the system, their hierarchy, connection to requirements, and their decomposition. It is used to ensure the internal flow of the system is covered in the tests.

# 3 Plan

This section describes the verification and validation processes that will be implemented. This includes the roles and responsibilities of the verification and validation team, the SRS verification plan, the design verification plan, the VnV plan verification plan, the implementation verification plan, the automated testing and verification tools, and the software validation plan.

## 3.1 Verification and Validation Team

The following table lists the role(s) and responsibilities for individuals working on the project's verification and validation.

Table 3: Verification and Validation Team

| **Name** | **Role(s)** | **Responsibilities** |
|---|---|---|
| Jacob Brodersen | VnV Systems Analyst | - Verify interoperability of system components<br>- System architecture and design verification and validation |
| Allen Tong | VnV Back-End Lead | - Verification of gameplay logic<br>- Verification of databases |
| Amos Edeaghe | VnV Webmaster | - Website content verification<br>- Website content validation |

*Continued on next page*

| Name | Role(s) | Responsibilities |
|---|---|---|
| Joshua Engel | VnV Creative Director and Team Lead | - Ensure consistency across subfields of work<br>- Verify traceability<br>- Validation of stakeholder requirements<br>- Coordinate document inspections<br>- Delegate verification and validation tasks |
| Nehan Munir | VnV Front-End Lead | - User input and output verification<br>- Rendering / projection verification |
| Dr. Paul Rapoport | Project Supervisor and Primary Stakeholder | - Work with Team Lead to validate the overall application and documentation |
| Lucas Dutton | Project TA and Primary Reviewer | - Provide feedback on verification and validation techniques (focus on coverage, effectiveness, and feasibility) |

## 3.2 SRS Verification

There are a few key approaches that will be used to verify the SRS.

First, ad hoc feedback from reviewers will be used. There will be three primary reviewer categories for the document: the project supervisor (Dr. Rapoport), peer reviewers (classmates), and Lucas (our TA). For the project supervisor, the SRS document will be provided ahead of a meeting in which the supervisor can bring up their concerns or recommendations for the document. In addition to accepting Dr. Rapoport's unstructured feedback,

we will ask targeted questions primarily focused around the functional and stakeholder requirements:

- Is there anything in particular that you found ambiguous or confusing?
- Do the Introduction and System Overview sections fully capture your intended problem, purpose, and scope?
- Do the stakeholder requirements accurately capture your project goals?
- Do the functional requirements support your vision for gameplay?
  - Are there any missing functional requirements that you would like to see added?

Note that Dr. Rapoport's work in the SRS is more validation-related than verification-related, but this review is still important, so it has been included here. Additionally, these questions still may prompt verification-related fixes.

For the peer reviewers, this will be an asynchronous process utilizing GitHub's issue tracker where the reviewers can read over the document on their own time and create formalized recommendations for improvement.

For Lucas, this will be an asynchronous process in the form of grading the SRS document. Lucas is very knowledgeable in SRS work and can focus on the more technical aspects of the document that the other reviewers may miss out on. This is a structured process as Lucas will be using the SRS grading rubric to evaluate the correctness of the SRS document.

Second, internal reviews will be performed. Initially, each team member works on their own section with limited interaction (other than ensuring consistency where work overlaps between sections). Once each team member has an initial draft of their work, another team member will be assigned to review their work. Once each team member has reviewed their assigned team member's work, we will hold a team meeting. In this team meeting, each team member will have a chance to voice their concerns/recommendations. The whole team will then give their input as to what course of action should follow.

SRS Verification Checklist:

- Every requirement is:
  - Abstract

  - Unambiguous
  - Traceable (clearly labelled and numbered)
  - Validatable
  - Verifiable
  - Complete
  - Consistent

- Each feature and goal identified through stakeholder elicitation has an associated requirement
- No conflicting requirements
- All assumptions are clearly defined
- All hazards and constraints are documented

Any matters identified during the SRS Verification process will be converted into GitHub issues. Once an issue is attended to, it will be reviewed again to ensure that it adheres to the guidelines laid out in this section before being resolved.

## 3.3 Design Verification

The design verification entails ensuring that the software's design is correct and complete. To do this, the following strategies will be used:

- Module Interface Specification (MIS) review by each team member as well as peer review by classmates:
  - Ensure that each requirement is traced to a module
  - Ensure that each module correctly adheres to the linked requirements
  - Ensure that all expected inputs and outputs of each module are clearly defined
  - Each module should follow SOLID principles
- System Architecture review by each team member as well as peer review by classmates:

- Ensure that subsystem responsibilities are well-separated
- Ensure architecture supports maintainability and extensibility
- Ensure that all major system components are represented
- Ensure that all interfaces are clearly defined (both between subcomponents as well as external interfaces)
- Ensure that the architecture supports all requirements
- Ensure that dependencies between subsystems are minimized and necessary

Any matters identified during the Design Verification process will be converted into GitHub issues. Once an issue is attended to, it will be reviewed again to ensure that it adheres to the guidelines laid out in this section before being resolved.

## 3.4 Verification and Validation Plan Verification

There are a few different methods that will be used for verifying the VnV plan.

First, each team member will review the document as a whole. Key goals of this review include ensuring consistency across the document and ensuring that all descriptions are concise and clear.

Second, mutation testing will be performed. Specifically, errors will purposefully be introduced (e.g., omitted requirements, incorrect expected outputs, etc.) to see if the VnV plan's review and testing methods would catch the error. This method provides confidence that the methods in this plan are adequate.

Third, peer review of the document will be conducted. This review process will happen through GitHub issues. Similar to the team member review, this will focus on consistency/unambiguity checks and clarity of the document.

Finally, TA feedback will be utilized. Due to the TA's extensive knowledge of VnV plans, this will allow for deeper analysis into whether the methods within the VnV plan are effective and efficient at verifying the requirements and ensuring sufficient test coverage.

VnV Checklist:

- Every requirement has at least one associated verification method

- Verification and validation activities are feasible with the available resources and timeline of the capstone
- All test cases are traceable to requirements
- Test coverage is sufficient to provide confidence in system correctness

Any matters identified during the VnV Plan process will be converted into GitHub issues. Once an issue is attended to, it will be reviewed again to ensure that it adheres to the guidelines laid out in this section before being resolved.

## 3.5 Implementation Verification

The below list outlines the methods used to verify the implementation:

- Run Pylint (Python) and ESLint (JavaScript) (see 3.6) to check for errors, style inconsistencies, etc.
- Measure code coverage for testing using Coverage.py/Pytest-cov (Python) and Jest (JavaScript) (see 3.6)
- Code walkthrough by each team member
  - Focus on logic errors throughout the code that could cause issues with functionality or performance
  - Reviews will be submitted through pull requests before merging on GitHub
- Peer code review through GitHub issues
  - Focus on general logic errors, maintainability concerns, performance concerns, and potential defects
- Automated test execution (see 3.6)
  - Ensure that each requirement (stakeholder, functional, data, and non-functional) has an associated system test-suite (defined in 4) (except for usability related requirements, see below)
  - Unit testing as described in 5

  - Automated execution of tests through GitHub Actions

- To verify non-functional usability requirements of the system, usability testing with users will be conducted (e.g., testing percentage of users that can perform a task without external assistance, see 6.2)

Any matters identified during the Implementation Verification process will be converted into GitHub issues. Once an issue is attended to, it will be reviewed again to ensure that it adheres to the guidelines laid out in this section before being resolved.

## 3.6 Automated Testing and Verification Tools

Below are the various tools our team plans to use for automated testing:

- Automated Unit and System Testing: Pytest (Python) **?** and Jest (JavaScript) **?**
- Build Verification and Automatic Test Execution for Pull Requests: GitHub Actions
- Linter for Static Analysis / Coding Standard Enforcement: Pylint (Python) **?** and ESLint (JavaScript) **?**
- Code Coverage for Testing: Coverage.py through Pytest-cov (Python) **?** and Jest (JavaScript) **?**
  - Code coverage metrics will be summarized as percentage of statements and branches covered as well as traceability of which test covered which statement/branch.
- Static Type Checking: Mypy (Python) **?** and TypeScript (superset for JavaScript) **?**
- Performance Measurement: cProfile **?**, Timeit **?**, and Tracemalloc **?** (Python) and Mitata **?** (JavaScript)

Note that these tools are consistent with the tools listed in the Expected Technology section of the Development Plan.

## 3.7 Software Validation

To validate the software, there are a few different methods.

First, our team will perform review sessions with our project supervisor, Dr. Rapoport. Dr. Rapoport is the main stakeholder of the project and therefore, review sessions will help to determine whether the software meets Dr. Rapoport's overall vision for the project. A main review session will be a demonstration with the Rev 0 product. This review will occur shortly after the scheduled Rev 0 demo for the class. During these review sessions, Dr. Rapoport will be tasked with validating whether the current version of the software aligns with the identified stakeholder requirements in the SRS and whether the software supports his overall product vision including gameplay and educational value.

Second, for a more detailed analysis of whether the software meets user needs, we will perform user testing sessions where users with the defined characteristics of our stakeholder groups (see SRS) will play through the game. While the test subjects are playing the game, our team will collect feedback from them on their experience. General validation of software usability will be successful if at least 80% of the participants are able to complete all core gameplay tasks without intervention and no major usability concerns are commonly identified during the sessions. Additionally, at the end of the session our team will ask structured questions about the user's experience. These questions are listed below:

- Which parts did you find most intuitive when using the software?
- Which parts did you find most frustrating and/or confusing when using the software?
- If you could change one part of the software, what would you change?
- If you could keep one part of the software, what would you keep?
- Are there any additional features you would like to see?

Third, our Custom Documents (CDs) also promote software validation. Our CD Fall is the User/Stakeholder/Client Interview Report. This document will allow us to clarify user/stakeholder/client goals and requirements and we can then cross-check these against the software specification. Our CD Winter is the Usability Report. This document can help to validate that

the software fits with the user's mental model of the gameplay and usability features. Note that these CDs are heavily related to, and have some overlap, with the first two methods, review sessions with Dr. Rapoport and user testing sessions.

Any matters identified during the Software Validation process will be converted into GitHub issues. Once an issue is attended to, it will be reviewed again to ensure that it adheres to the guidelines laid out in this section before being resolved.

# 4 System Tests

## 4.1 Tests for Functional Requirements

The tests in this section verify the functional requirements in Section 6.2 of the SRS. Each test covers a group of related requirements, and the tests are grouped into the same areas as the requirements: round setup, tasks and answer checking, scoring, saving and recovery, 3D visualization, guided learning, accounts and competitive play, and wellbeing. Requirements are referred to by their numbers in Section 6.2 of the SRS (for example, FR9). Section 4.3 shows which tests cover which requirements. Tests marked Automatic are run by a script. Tests marked Manual are performed by a tester who checks the result on screen. Expected values come from Dr. Rapoport's rules documents and from the SRS.

### 4.1.1 Round Setup

These tests cover FR1 to FR6.

1. ST-FR-1

   Control: Automatic

   Initial State: The game is loaded and no round is in progress.

   Input: The tester presses START and then, without choosing a scoring base, tries to press START again. The tester then chooses the octal base and presses START.

   Output: After the first START, 512 cells are displayed with their numbers hidden. The starter and timer do not appear until a base is chosen.

After the base is chosen and START is pressed, one starter number is shown in its cell with a white background, and a timer showing elapsed seconds begins counting.

Test Case Derivation: The cube has 8 x 8 x 8 = 512 cells (FR1). A base must be chosen before the starter appears (FR4, FR5).

How test will be performed: A script drives the interface, counts the displayed cells, checks that none shows a number, and reads the starter cell and timer at each step.

2. ST-FR-2

   Control: Automatic

   Initial State: The game is loaded, with the random number generator replaced by a seeded generator.

   Input: 20,000 rounds are started.

   Output: All three plane colours (yellow, green, blue) and all 384 orientations occur at least once, and no round uses a plane colour or orientation outside these sets.

   Test Case Derivation: FR2 and FR3 require a random choice from 3 colours and 384 orientations. With 20,000 rounds, each orientation is expected about 52 times, so a missing orientation would indicate a defect and not chance.

   How test will be performed: A script starts the rounds, records the plane colour and orientation of each, and compares the sets observed against the allowed sets.

3. ST-FR-3

   Control: Automatic

   Initial State: A new player with an empty used-starter list.

   Input: 192 rounds are played, recording the starter of each, followed by a 193rd round.

   Output: The 192 starters are all different. After the 192nd round, the list is cleared, so the 193rd starter may repeat an earlier starter.

Test Case Derivation: FR6 says no starter is reused until 192 rounds have been played, then the list is cleared.

How test will be performed: A script plays the rounds in a loop and checks for duplicates among the first 192 starters and for the cleared list afterward.

### 4.1.2 Tasks and Answer Checking

These tests cover FR7 to FR26 and FR30.

1. ST-FR-4

   Control: Automatic

   Initial State: Task 1 is awaiting a cell selection. The starter is at position (0,0,0).

   Input: The tester selects 20 different incorrect cells, then selects the target cell, then selects the target cell again. In a separate run, the tester clicks a cell while no selection is expected (for example, while the Addition Box is shown).

   Output: Every incorrect cell turns red and the round continues with no limit or lock-out. The target cell turns white. The second selection of the white cell is ignored. The click made while no selection is expected changes nothing.

   Test Case Derivation: FR8 gives white for target cells and red for others, with no limit on incorrect selections, and ignores cells that are already white and selections made while none is expected. The target for task 1 is the cell whose x, y, and z coordinates each differ by 4 modulo 8, which is (4,4,4) (FR9).

   How test will be performed: A script clicks each cell and reads its colour after each click.

2. ST-FR-5

   Control: Automatic

Initial State: A fixed cube array, a fixed plane colour, and a fixed starter for each of the tasks 1 to 8. For tasks 4 to 7, one setup uses a continuous line and another uses a split line.

Input: For each task, the tester selects every cell of the expected line, in any order. For tasks 4 to 7, this is done once for a continuous line and once for a split line. For task 8, the tester also selects the triagonal used in task 7.

Output: Each expected target cell turns white and no other cell does. The number of target cells is 1 for task 1, 3 for task 2, and 7 for tasks 3 to 8. Continuous and split lines are both accepted for tasks 4 to 7. In task 8, the triagonal from task 7 is not accepted. Across a round, the tasks appear in order from 1 to 8 and no two tasks use the same target cells.

Test Case Derivation: The expected cells for each task come from the task definitions (FR9 to FR16) in Dr. Rapoport's rules, worked out by hand for each fixed setup and stored with the test. FR7 requires the eight tasks in order and no repeated target cells.

How test will be performed: A script loads each fixed setup, selects the stored cells, and checks cell colours and counts. A second script generates 1,000 rounds and compares the target sets of all task pairs in each round.

3. ST-FR-6

Control: Automatic

Initial State: A task 2 setup in octal with starter 1 and the first additional number in the line equal to 2. A second setup is at the Subtraction Box for each of the cases in the table below.

Input: The tester tries to choose 0, 1, 2, and 3 additional numbers for task 2, and 0, 1, 5, and 6 for task 3. The tester then chooses 1 additional number for task 2 and enters 4 (incorrect) and then 3 (correct) in the Addition Box, pressing DONE each time. At the Subtraction Box, the tester enters a wrong answer and then the correct answer for each case in the table.

Output: 1 to 2 additional numbers are accepted for task 2 and 1 to 5 for tasks 3 to 8, and other values are rejected. After a valid choice,

the starter and exactly that many additional numbers are shown in a column. The answer 4 is cleared and the box turns red, and the answer 3 is accepted and the additional number appears in white. A wrong subtraction answer is cleared and the box turns red. The correct answer in each case is accepted, the timer stops, and the remaining cells of the line are shown in white (for task 1, the answer is shown in the complementary cell).

Test Case Derivation: FR17 gives the allowed ranges and FR18 gives the column. $1 + 2 = 3$ in octal (FR19). FR20 gives the constants 1001 for task 1, 2002 for task 2, and 4004 for tasks 3 to 8, and the expected answer is the constant minus the starter or sum, in octal (FR21).

How test will be performed: A script enters each value, reads the box colour, box contents, and cell colours, and compares the accepted subtraction answer with an independent octal subtraction function.

All values in the table below are octal.

| **Task** | **Starter or sum** | **Constant** | **Answer** |
|---|---|---|---|
| 1 | 25 | 1001 | 754 |
| 2 | 700 | 2002 | 1102 |
| 5 | 1234 | 4004 | 2550 |

4. ST-FR-7

   Control: Automatic

   Initial State: The Subtraction Box is shown. A round is awaiting a cell selection in a separate run.

   Input: The tester presses DONE with an empty box, then enters "8", "abc", and "7a", pressing DONE each time. In the separate run, the tester presses a key that has no function in the current state (for example, F9).

   Output: Each invalid answer shows a message asking for a valid answer and is not counted as an incorrect answer, so the count of incorrect attempts does not change. The unused key shows a message saying

the input was not recognized, and the round state and timer do not change.

Test Case Derivation: FR22 allows only the octal digits 0 to 7 and says invalid input must not count as incorrect. FR23 requires a message for unrecognized input.

How test will be performed: A script enters each value or key press and compares the message and the incorrect-attempt counter before and after.

5. ST-FR-8

   Control: Automatic

   Initial State: Task 3 has just been completed, with the white cells and question on screen and the timer at 40 seconds. In separate runs, a round is at the Subtraction Box of task 8, and a round is in progress at task 4 with a logged-in user.

   Input: The tester presses START to begin task 4. In the second run, the tester enters the correct answer to the task 8 subtraction. In the third run, the tester presses STOP GAME.

   Output: The cells of task 3 other than the starter return to their original colour, the previous question and answers are cleared, and the timer continues from 40 seconds. After task 8, the round ends, the final score is displayed, and a new round can be started. After STOP GAME, the round ends, all progress is erased, no score is recorded in statistics or leaderboards, and the game returns to its ready state.

   Test Case Derivation: FR24, FR25, and FR30 specify these outcomes.

   How test will be performed: A script compares cell colours, question text, and timer before and after each action, and checks the statistics and leaderboard records after STOP GAME.

6. ST-FR-9

   Control: Automatic

   Initial State: The cube array is loaded with one value altered, so one pair of cells that should sum to 1001 (octal) does not.

Input: The tester starts a round that would use the altered pair.

Output: The system does not present the task, generates a new setup, and writes the failure to the log.

Test Case Derivation: FR26 requires the sums and expected answers to be checked before each task, with a new setup and a log entry if a check fails.

How test will be performed: A script alters the array, starts the round, and checks that the task is replaced and that a log entry exists.

### 4.1.3 Scoring

These tests cover FR27 to FR29.

1. ST-FR-10

   Control: Automatic

   Initial State: A task is complete in the stated scoring base with the stated number of additional numbers, and the time spent on the task is as stated in the table.

   Input: The tester answers the subtraction correctly in each case.

   Output: The task score is as listed in the table.

   Test Case Derivation: FR27 gives the point values: decimal 10, 20, 30, 30, 40, 40, 60, 60 for tasks 1 to 8, and octal 14, 30, 44, 44, 60, 60, 110, 110, which are 12, 24, 36, 36, 48, 48, 72, 72 in decimal. Each additional number adds 10 (decimal) or 14 (octal, 12 in decimal). FR28 gives score = basic score x 1350 / t, rounded to the nearest whole number. For example, task 3 in decimal at 30 s is 30 x 1350 / 30 = 1350. Task 1 at 7 s is 10 x 1350 / 7 = 1928.57, which rounds to 1929.

   How test will be performed: A script sets the timer to the stated time, completes each task, and compares the score shown with the value from an independent calculation.

| Task | Base | Additional numbers | Time | Task score |
|---|---|---|---|---|
| 1 | Decimal | 0 | 7 s | 1929 |
| 1 | Decimal | 0 | 30 s | 450 |
| 1 | Octal | 0 | 30 s | 540 |
| 2 | Decimal | 2 | 30 s | 1800 |
| 3 | Decimal | 0 | 30 s | 1350 |
| 3 | Octal | 0 | 30 s | 1620 |
| 8 | Decimal | 0 | 30 s | 2700 |
| 8 | Octal | 0 | 30 s | 3240 |

2. ST-FR-11

   Control: Automatic

   Initial State: A full round in decimal with a fixed time of 30 seconds per task and no additional numbers.

   Input: The tester completes all eight tasks, giving one incorrect answer along the way.

   Output: The running total after each task equals the sum of the task scores so far. The final score is (10 + 20 + 30 + 30 + 40 + 40 + 60 + 60) x 1350 / 30 = 290 x 45 = 13,050. The incorrect answer does not change the total.

   Test Case Derivation: FR29 says the total is displayed after each task and the final score is the sum of the eight timed scores.

   How test will be performed: A script plays the round and reads the total after each task.

#### 4.1.4 Saving and Recovery

These tests cover FR31 and FR32.

1. ST-FR-12

Control: Automatic

Initial State: A round is in progress at task 3 with the timer at 25 seconds.

Input: The tester (1) reloads the page and chooses to resume. (2) With the save location made to fail, makes a move that changes the round state. (3) Alters a stored saved round, then opens the page.

Output: (1) The round resumes at task 3 with the same cell colours and the timer at its saved value. (2) The system tries to save 3 times, then tells the user that saving failed, and the round continues. (3) The system detects that the saved round is invalid, discards it, and tells the user.

Test Case Derivation: FR31 saves the round at every state change, retries up to 3 times, and reports failure. FR32 restores a valid saved round and discards an invalid one.

How test will be performed: A script plays to this point, reloads the page, and compares the restored state. It then makes the save call fail and counts the attempts and messages, then edits one stored value, reloads, and checks for the message and for an empty saved state.

#### 4.1.5 3D Visualization

These tests cover FR33 to FR37.

1. ST-FR-13

   Control: Manual

   Initial State: A round is in progress at task 2 with a few cells already selected.

   Input: The tester switches from the slice display to the 3D view and back. In the 3D view, the tester rotates and zooms the cube with the mouse, then with the keyboard only. The tester then chooses a layer along each of the three axes, steps to the adjacent layer, and restores all layers.

   Output: Each number is in the same cell in both views, selected cells show the same colours, the round state does not change, and the timer

keeps running. The cube rotates and zooms with both input methods. Only the chosen layer is shown clearly with the other layers faded or hidden, the chosen layer and the selected cells are highlighted in visibly different ways, stepping moves to the adjacent layer, and restoring shows all layers.

Test Case Derivation: FR33 and FR36 require consistency between the views and no pause in the timer. FR34 requires pointer and keyboard control. FR35 describes the layer controls.

How test will be performed: The tester compares 10 cells chosen across the cube in both views, notes the timer before and after each switch, and repeats the layer steps for each axis.

2. ST-FR-14

   Control: Manual

   Initial State: The 3D view is shown on a device with reduced performance (for example, a browser with its CPU throttled).

   Input: The tester keeps the 3D view open until its performance falls below the minimum level in the Non-Functional Requirements.

   Output: The system warns the user and offers to switch to the slice display. Accepting returns the user to the slice display with the round state unchanged.

   Test Case Derivation: FR37. The minimum level is set in the Non-Functional Requirements.

   How test will be performed: The tester throttles the browser using its developer tools and observes the warning.

#### 4.1.6 Guided Learning and Tutorial

These tests cover FR38 to FR41.

1. ST-FR-15

   Control: Manual

   Initial State: The user opens the tutorial.

Input: The tester completes the tutorial, then replays it, then exits it part way through.

Output: The tutorial includes practice for all eight task types, shows only instructions that match the current step, and can be skipped, exited, or replayed at any time. Nothing from the tutorial appears in statistics or leaderboards.

Test Case Derivation: FR38.

How test will be performed: The tester follows the tutorial, checks the instruction for each step, and checks the statistics page afterward.

2. ST-FR-16

   Control: Automatic

   Initial State: Task 3 is awaiting a cell selection.

   Input: The tester selects an incorrect cell, requests a hint, and then tries to request the solution after 0, 2, and 3 incorrect selections. In a separate run, the tester gives 3 incorrect answers to the same addition.

   Output: After the incorrect cell, a message names the type of line being sought and the rule it follows, without naming a cell. The hint states the coordinate step (for example, "each coordinate differs by 4") without identifying a cell, and the request is recorded. The solution can be requested only after 3 incorrect attempts at the same step. For cell selection, the correct cells are highlighted one after another in the order the line is traced. For an addition, the worked octal calculation is shown. Each request is recorded.

   Test Case Derivation: FR39, FR40, and FR41.

   How test will be performed: A script makes the attempts, reads each message, checks that none contains the number or coordinates of a target cell, and checks when the solution option becomes available.

### 4.1.7 Accounts and Competitive Play

These tests cover FR42 to FR53.

1. ST-FR-17

Control: Automatic

Initial State: No user is logged in and network traffic is being recorded. The account creation page is available.

Input: The tester plays a full round as a guest. The tester then tries each of the account values in the table below.

Output: The guest round completes and no data about it is sent to the online service. Values marked Accept are accepted after the user confirms the entered username and email address. Values marked Reject are rejected with a message giving the reason.

Test Case Derivation: FR42 says no guest data is sent online. FR43 requires a unique username, a unique valid email, and a password of at least 8 characters. FR44 requires 3 to 20 characters and no prohibited terms. The boundary values 2, 3, 20, and 21 characters test each limit.

How test will be performed: A script plays the round and inspects the recorded requests, then submits each account case and reads the response. A test account already registered is used for the duplicate cases.

| **Case** | **Value** | **Expected** |
|---|---|---|
| Username, 2 characters | ab | Reject |
| Username, 3 characters | abc | Accept |
| Username, 20 characters | 20 letters | Accept |
| Username, 21 characters | 21 letters | Reject |
| Username with prohibited term | a term from the list | Reject |
| Duplicate username | an existing name | Reject |
| Email without “@” | user.example.com | Reject |
| Duplicate email | an existing email | Reject |
| Password, 7 characters | Abcdef1 | Reject |
| Password, 8 characters | Abcdefg1 | Accept |

2. ST-FR-18

Control: Manual

Initial State: One registered account with a test email inbox.

Input: The tester logs in with the correct credentials and logs out, then tries a wrong password with a real username and a username that does not exist. The tester then requests a password reset for the registered address and for an address that is not registered.

Output: Correct credentials log the user in and out. Both login failures show the same message, which does not say whether the username or password was wrong. A reset link arrives only for the registered address, and the page shows the same message in both reset cases.

Test Case Derivation: FR45 and FR46.

How test will be performed: The tester submits each case, compares the two failure messages, and checks the inbox and the page text.

3. ST-FR-19

Control: Automatic

Initial State: A logged-in user with no completed rounds. A leaderboard for decimal scoring holds 10 entries.

Input: The user completes two rounds in decimal with final scores 10,000 and 12,000 and times 300 s and 280 s. One round scores higher than the lowest leaderboard entry and one scores lower than all entries. The tester then sends a request directly to the service to add a score of 99,999 without playing a round.

Output: Each completed round is recorded with its score, base, total time, and date. The statistics show 2 completed rounds, a highest score of 12,000, an average of 11,000, and a fastest time of 280 s for decimal, and no data for octal. The higher score appears on the leaderboard within 1 minute and the lowest entry is removed, the lower score does not appear, only username and score or time are shown, and octal and decimal boards are separate. The direct request is rejected and no record is created or changed.

Test Case Derivation: FR47, FR48, and FR49. The average is (10,000 + 12,000) / 2 = 11,000. FR50 allows records to be created only from rounds the system has completed.

How test will be performed: A script plays the rounds with a controlled timer, reads the statistics page, polls the leaderboard, and checks the records before and after the direct request.

4. ST-FR-20

   Control: Automatic

   Initial State: Two different users are logged in on the same day.

   Input: Both open the daily challenge and one completes it. The tester then copies the shared summary. The system date is moved forward one day and the daily challenge is opened again.

   Output: Both users get the same plane colour, orientation, and starter, and a leaderboard of that day's results is available. The summary contains the date, final score, and total time, and no cell number or answer. On the next day the setup is different.

   Test Case Derivation: FR51 and FR52.

   How test will be performed: A script reads the setup for both users, searches the copied text for any cell value or answer from the round, and reads the setup again after the date changes.

5. ST-FR-21

   Control: Automatic

   Initial State: A logged-in user with 3 recorded rounds and entries on the leaderboard.

   Input: The user deletes their account.

   Output: The username, email, credentials, scores, statistics, and leaderboard entries are removed, and logging in again fails.

   Test Case Derivation: FR53.

   How test will be performed: A script deletes the account and then queries each place the data was stored.

### 4.1.8 Wellbeing

These tests cover FR54 and FR55.

1. ST-FR-22

   Control: Automatic

   Initial State: A session is in progress with a round at task 2 and the timer at 20 seconds. The session clock can be advanced. The 3D view is shown with animations on.

   Input: The tester advances the session clock by one hour and then two hours. The tester then turns animations off, rotates the cube, and selects a cell.

   Output: A reminder showing how long the user has been playing appears after each hour, and the round state and round timer are unchanged. With animations off, the cube rotates without animation and cell feedback has no visual effects, and the change applies immediately with no reload.

   Test Case Derivation: FR54 and FR55.

   How test will be performed: A script advances a mocked clock and reads the reminder text, then compares the round state and timer before and after. The animation check is made by the tester on screen.

## 4.2 Tests for Nonfunctional Requirements

The following tests evaluate the nonfunctional requirements defined in the Software Requirements Specification (SRS). Testing will assess the performance, usability, maintainability, portability, reliability, security, privacy, and accessibility of the Magic-8 application.

Performance tests will measure response times under a defined environment. Usability tests will involve participants who have not previously used magic-8. Maintainability requirements will be evaluated through code inspections and automated tests. Reliability, security, and accessibility will be assessed through automated and manual testing.

No accuracy nonfunctional requirements were defined in the SRS. Mathematical correctness will instead be evaluated through the corresponding functional tests in Section 4.1.

### 4.2.1 Performance Testing

**Cube Manipulation Performance**

1. NFR-T1: Cube Rotation Response Time

   Type: Nonfunctional, Dynamic, Automated.

   Initial State: The application is running in the reference test environment, and a state containing a fully initialized three-dimensional cube is open.

   Input/Condition: A series of supported cube manipulation inputs, including horizontal and vertical rotations, as well as choosing a plane of the cube.

   Output/Result: The visible cube orientation updates within 100 milliseconds for at least 95% of the measured rotation inputs.

   How test will be performed: An automated test will simulate supported rotation inputs and record the time between each input being received and the corresponding visible orientation update. The test will be repeated for a sufficient number of rotation inputs to get a reliable measurement. The percentage of updates completed within 100 milliseconds will be calculated and compared against the 95% requirement.

2. NFR-T2: Cell-Value Editing Response Time

   Type: Nonfunctional, Dynamic, Automated.

   Initial State: The application is running in the reference test environment, and the state has an editable cell selected.

   Input: A series of valid cell-value edits submitted through the supported input controls.

   Output: Accepted cell-value edits are visibly reflected within 100 milliseconds for at least 95% of measured edits.

   How test will be performed: An automated test will submit valid cell-value edits and record the time between submission and the visible update of the corresponding cell. The test will be repeated across multiple cells and values. The percentage of accepted edits reflected

within 100 milliseconds will be calculated and compared against the 95% requirement.

### 4.2.2 Usability Testing

**First-Time User Interaction**

1. NFR-T3: Puzzle Opening Usability

   Type: Nonfunctional, Dynamic, Manual, User Evaluation.

   Initial State: The application is open on its initial screen, and the participant has not previously used Magic-8 or received instructions on how to operate it.

   Input/Condition: The participant is asked to locate and open a puzzle.

   Output/Result: At least 80% of participants successfully open a puzzle without external assistance or prior training.

   How test will be performed: Participants will be given the application and asked to open a puzzle. The evaluator will observe whether each participant completes the task independently. The number of successful participants will be recorded and divided by the total number of participants. The resulting percentage will be compared against the 80% requirement.

2. NFR-T4: Cube Manipulation Usability

   Type: Nonfunctional, Dynamic, Manual, User Evaluation.

   Initial State: A puzzle is open, and the three-dimensional cube is visible to the participant.

   Input/Condition: The participant is asked to rotate the cube using the available rotation controls to view different sides and orientations.

   Output/Result: At least 80% of participants successfully complete each defined cube-rotation task without external assistance.

   How test will be performed: Participants will be given several rotation tasks that require them to change the orientation of the three-dimensional cube. The evaluator will record whether each task is com-

pleted independently and correctly. The success rate for each defined task will be calculated and compared against the 80% requirement.

3. NFR-T5: Cell Selection Usability

   Type: Nonfunctional, Dynamic, Manual, User Evaluation.

   Initial State: A puzzle is open, and the participant can interact with the visible cube.

   Input/Condition: The participant is asked to select specific cells identified by their positions within the cube.

   Output/Result: At least 80% of participants correctly select the requested cells without external assistance.

   How test will be performed: Participants will be given a series of cell-selection tasks. The evaluator will record whether the correct cell is selected for each task without assistance. The percentage of participants successfully completing each defined task will be calculated and compared against the 80% requirement.

4. NFR-T6: Cell-Value Editing Usability

   Type: Nonfunctional, Dynamic, Manual, User Evaluation.

   Initial State: A puzzle is open, and the participant has selected a cell that allows its value to be edited.

   Input/Condition: The participant is asked to enter and submit a valid value for the selected cell.

   Output/Result: At least 80% of participants successfully enter and submit a valid cell value without external assistance.

   How test will be performed: Participants will be asked to select a cell, enter a specified valid value, and submit the change. The evaluator will record whether the participant successfully completes the task independently. The percentage of successful participants will be calculated and compared against the 80% requirement.

Along with these tests participants may be asked to describe any difficulties they had with the application for further development.

### 4.2.3 Maintainability Testing

**Code Organization and Independent Testing** The maintainability requirements will be evaluated through static inspection and automated testing rather than conventional user-based tests.

1. NFR-T7: Separation of System Components

   Type: Nonfunctional, Static, Manual, Code Inspection.

   Initial State: The completed application source code and its accompanying documentation are available for inspection.

   Input/Condition: The source code is reviewed to identify the organization of mathematical logic, user-interface logic, three-dimensional visualization logic, and online storage interactions.

   Output/Result: Four identifiable components responsible for the specified areas are found, and each component has documented responsibilities.

   How test will be performed: Members of the development team will inspect the source code and supporting documentation. The inspection will identify the components responsible for each of the four specified areas and verify that their responsibilities are documented. The results will be recorded in an inspection checklist.

2. NFR-T8: Independent Mathematical Logic Testing

   Type: Nonfunctional, Dynamic, Automated.

   Initial State: The automated test suite for the core mathematical operations is available.

   Input/Condition: The mathematical test suite is executed without launching the user interface or rendering the three-dimensional cube.

   Output/Result: The mathematical test suite executes successfully without requiring the user interface or three-dimensional visualization.

   How test will be performed: The mathematical test suite will be executed independently from the application interface and visualization components. The test execution process and results will be inspected

to confirm that the mathematical operations can be tested without launching either component.

### 4.2.4 Portability Testing

**Browser Compatibility and Deployment**

1. NFR-T9: Browser Compatibility

   Type: Nonfunctional, Dynamic, Manual and Automated.

   Initial State: The application is deployed and accessible through the chosen supported browsers.

   Input/Condition: The core functional test suite is executed using each supported browser version.

   Output/Result: All core functional tests pass on every chosen browser and version.

   How test will be performed: The core functional test suite from Section 4.1 will be executed separately on each supported browser. The results will be recorded for each browser and compared to identify any differences in functionality or behaviour. The requirement is satisfied when the core functional test suite passes on every chosen browser.

2. NFR-T10: Browser-Based Deployment

   Type: Nonfunctional, Dynamic, Manual.

   Initial State: A clean reference device is available with a supported desktop or laptop web browser installed. Python and any standalone Magic-8 application have not been installed.

   Input/Condition: The tester accesses the deployed Magic-8 application through the supported web browser.

   Output/Result: The application launches and can be used without installing Python or a standalone desktop application.

   How test will be performed: A clean reference device will be used to access the deployed application through a supported browser. The tester will then open the application and perform basic interactions.

### 4.2.5 Reliability Testing

**Puzzle State and Failure Handling**

1. NFR-T11: Puzzle-State Integrity

   Type: Nonfunctional, Dynamic, Automated.

   Initial State: A valid puzzle state has been initialized.

   Input/Condition: A sequence of supported cube manipulations, cell selections, and accepted cell-value edits is performed.

   Output/Result: The puzzle state remains valid after every supported state update and continues to satisfy the state-validity rules.

   How test will be performed: Automated tests will perform sequences of cube manipulations, cell selections, and valid cell-value edits. After each operation, the resulting puzzle state will be checked against the state-validity rules. This will verify no error states are met through supported intputs.

2. NFR-T12: Invalid Input Handling

   Type: Nonfunctional, Dynamic, Automated.

   Initial State: A valid puzzle is open, and a cell-value input control is available.

   Input: Invalid cell-value inputs, including unsupported numerical values, values outside the permitted range, and incorrectly formatted inputs.

   Output: Invalid inputs are rejected, the puzzle state remains valid, and the application continues operating without terminating.

   How test will be performed: Automated tests will submit each defined category of invalid input through the supported cell-editing controls. After each submission, the resulting puzzle state will be checked to ensure it is still valid.

3. NFR-T13: Online Operation Failure Handling

   Type: Nonfunctional, Dynamic, Manual and Automated.

Initial State: The application is running with online storage functionality available.

Input/Condition: Online storage operations are attempted while simulating failures such as a lost network connection, unavailable server, or rejected storage request.

Output/Result: The application displays an appropriate failure message when unsuccessful.

How test will be performed: The online storage operations will be tested under simulated failure conditions. Network access may be interrupted or server responses may be configured to return failure results. The tester will verify that the application displays an appropriate failure message for each simulated failure and does not incorrectly indicate that the operation was successful.

### 4.2.6 Security and Privacy Testing

**Network Security and User Data Protection**

1. NFR-T14: Network Communication Security

   Type: Nonfunctional, Dynamic, Manual, Security Testing.

   Initial State: The application is deployed with account functionality and network communication enabled.

   Input/Condition: Account creation, login, and operations involving user data are performed while monitoring network requests.

   Output/Result: All network requests involving account credentials or user data use HTTPS with valid TLS. None of which is transmitted through unencrypted HTTP requests.

   How test will be performed: A network inspection tool will be used to monitor requests generated during account creation, login, and user-data operations. The protocol used for each relevant request will be inspected, and TLS validity will be verified. Also checking no data is not transmitted through unencrypted HTTP requests.

2. NFR-T15: User Data Isolation

Type: Nonfunctional, Dynamic, Automated, Security Testing.

Initial State: At least two separate test accounts have been created, each containing private stored data.

Input/Condition: One authenticated test account attempts to retrieve, modify, or delete private data belonging to the other test account.

Output/Result: The system prevents the authenticated user from accessing or modifying another user's private stored data.

How test will be performed: Automated authorization tests will authenticate using one test account and attempt to access private data belonging to the second account through every supported operation. Attempts to retrieve, modify, or delete the other account's data will be recorded.

3. NFR-T16: Authentication Secret Protection

   Type: Nonfunctional, Static, Manual, Security Inspection.

   Initial State: The application source code and application-controlled persistent storage are available for inspection.

   Input/Condition: Authentication-related information is examined in the application's persistent storage and relevant storage operations.

   Output/Result: No authentication secrets are found stored in plaintext within application-controlled persistent storage.

   How test will be performed: The development team will inspect the application's persistent storage, including relevant local storage and database records. Checking if any authentication-related values are stored in plaintext. The inspection results will be documented.

4. NFR-T17: Personal Data Minimization

   Type: Nonfunctional, Static, Manual, Privacy Inspection.

   Initial State: The application's data fields, collection operations, and persistent storage structures are available for inspection.

   Input/Condition: All personal information collected or retained by the application is reviewed alongside its documented functional purpose.

Output/Result: Every collected or retained personal information field has a documented purpose related to the application's implemented functionality.

How test will be performed: The development team will inspect the application's data fields, account information requirements, collection operations, and persistent storage. Each personal information field will be compared against the functionality that requires it. Any field without a documented functional purpose will be identified and reviewed for removal.

### 4.2.7 Accessibility Testing

**Keyboard Accessibility and Visual Contrast**

1. NFR-T18: Keyboard Accessibility

   Type: Nonfunctional, Dynamic, Manual.

   Initial State: The application is open, and a puzzle is available for interaction.

   Input/Condition: The tester uses only a keyboard to navigate the application and operate essential puzzle controls, including puzzle navigation, cell selection, and cell-value editing.

   Output/Result: All essential puzzle controls can be accessed and operated using a keyboard without requiring pointer interaction.

   How test will be performed: The tester will disable pointer interaction and use keyboard inputs to navigate the application. Each essential puzzle control will be accessed and operated using the supported keyboard controls.

2. NFR-T19: Visual Contrast

   Type: Nonfunctional, Static, Automated and Manual, Accessibility Inspection.

   Initial State: The application interface is available for inspection.

   Input/Condition: The contrast between essential interface text and controls and their adjacent backgrounds is measured.

Output/Result: Every essential interface text element and control meets a minimum contrast ratio of 3:1 against its adjacent background.

How test will be performed: An automated or manual contrast assessment tool will be used to inspect the application's essential interface text and controls. The contrast ratio of each element against its adjacent background will be recorded. Any element with a contrast ratio below 3:1 will be identified for correction.

## 4.3 Traceability Between Test Cases and Requirements

Table 4 shows which system tests in Section 4.1 support each functional requirement in the SRS.

| **Requirement** | **Tests** |
|---|---|
| FR1 | ST-FR-1 |
| FR2 | ST-FR-2 |
| FR3 | ST-FR-2 |
| FR4 | ST-FR-1 |
| FR5 | ST-FR-1 |
| FR6 | ST-FR-3 |
| FR7 | ST-FR-5 |
| FR8 | ST-FR-4 |
| FR9 | ST-FR-4, ST-FR-5 |
| FR10 to FR16 | ST-FR-5 |
| FR17 | ST-FR-6 |
| FR18 | ST-FR-6 |
| FR19 | ST-FR-6 |
| FR20 | ST-FR-6 |
| FR21 | ST-FR-6 |
| FR22 | ST-FR-7 |
| FR23 | ST-FR-7 |

| Requirement | Tests |
|---|---|
| FR24 | ST-FR-8 |
| FR25 | ST-FR-8 |
| FR26 | ST-FR-9 |
| FR27 | ST-FR-10 |
| FR28 | ST-FR-10 |
| FR29 | ST-FR-11 |
| FR30 | ST-FR-8 |
| FR31 | ST-FR-12 |
| FR32 | ST-FR-12 |
| FR33 | ST-FR-13 |
| FR34 | ST-FR-13 |
| FR35 | ST-FR-13 |
| FR36 | ST-FR-13 |
| FR37 | ST-FR-14 |
| FR38 | ST-FR-15 |
| FR39 | ST-FR-16 |
| FR40 | ST-FR-16 |
| FR41 | ST-FR-16 |
| FR42 | ST-FR-17 |
| FR43 | ST-FR-17 |
| FR44 | ST-FR-17 |
| FR45 | ST-FR-18 |
| FR46 | ST-FR-18 |
| FR47 | ST-FR-19 |
| FR48 | ST-FR-19 |
| FR49 | ST-FR-19 |
| FR50 | ST-FR-19 |

| Requirement | Tests |
|---|---|
| FR51 | ST-FR-20 |
| FR52 | ST-FR-20 |
| FR53 | ST-FR-21 |
| FR54 | ST-FR-22 |
| FR55 | ST-FR-22 |

Table 4: Traceability Between Functional Requirements and System Tests

Table 5 provides the traceability between the nonfunctional test cases defined in Section 4.2 and their corresponding nonfunctional requirements in the SRS. Each requirement is associated with at least one test case that evaluates whether its specified criteria have been met.

| Test Case ID | Test Case Description | Requirement Verified |
|---|---|---|
| NFR-T1 | Cube Rotation Response Time | NFR_1: Cube Manipulation Performance |
| NFR-T2 | Cell-Value Editing Response Time | NFR_2: Cell-Value Editing Performance |
| NFR-T3 | Puzzle Opening Usability | NFR_3: Puzzle Opening Usability |
| NFR-T4 | Cube Manipulation Usability | NFR_4: Cube Manipulation Usability |
| NFR-T5 | Cell Selection Usability | NFR_5: Cell Selection Usability |
| NFR-T6 | Cell-Value Editing Usability | NFR_6: Cell-Value Editing Usability |
| NFR-T7 | Separation of System Components | NFR_7: Separation of System Components |

| | | |
|---|---|---|
| NFR-T8 | Independent Mathematical Logic Testing | NFR_8: Independent Testing of Mathematical Logic |
| NFR-T9 | Browser Compatibility | NFR_9: Browser Compatibility |
| NFR-T10 | Browser-Based Deployment | NFR_10: Browser-Based Deployment |
| NFR-T11 | Puzzle-State Integrity | NFR_11: Puzzle-State Integrity |
| NFR-T12 | Invalid Input Handling | NFR_12: Invalid Input Handling |
| NFR-T13 | Online Operation Failure Handling | NFR_13: Online Operation Failure Handling |
| NFR-T14 | Network Communication Security | NFR_14: Network Communication Security |
| NFR-T15 | User Data Isolation | NFR_15: User Data Isolation |
| NFR-T16 | Authentication Secret Protection | NFR_16: Authentication Secret Protection |
| NFR-T17 | Personal Data Minimization | NFR_17: Personal Data Minimization |
| NFR-T18 | Keyboard Accessibility | NFR_18: Keyboard Accessibility |
| NFR-T19 | Visual Contrast | NFR_19: Visual Contrast |

Table 5: Traceability Between Nonfunctional Test Cases and Requirements

# 5 Unit Test Description

## 5.1 Unit Testing Scope

## 5.2 Tests for Functional Requirements

### 5.2.1 Module 1

1. test-id1

   Type:
   Initial State:
   Input:
   Output:
   Test Case Derivation:
   How test will be performed:

2. test-id2

   Type:
   Initial State:
   Input:
   Output:
   Test Case Derivation:
   How test will be performed:

3. ...

### 5.2.2 Module 2

...

## 5.3 Tests for Nonfunctional Requirements

### 5.3.1 Module ?

...

## 5.4 Traceability Between Test Cases and Modules

# 6 Appendix

## 6.1 Symbolic Parameters

We did not have any symbolic parameters.

## 6.2 Usability Survey Questions?

During the semi-structured interview with the users, these quetions will serve as stress points to drive the interview when it reaches a lull. While these questions are deeply connected to the SRS documentation, the interviewers are expected to come up with questions on the spot based on immediate feedback they received during the tech demo.

1. **Menu Navigation:** Can you navigate to starting a game and view the leaderboard at the end? ( 1-5 scale: 1 = Very Easily, 4 = Not Without Help, 5 = Still Confused)

   **Rationale:** This question serves as a surrogate for Non-Functional Req 19 (Visual Contrast) based on the capabilities of the user being asked the question as those with visual impairments will struggle to complete the tutorial without high visual contrast and following accessibility guidelines. This is directly related to Non-Functional Req 18 (Keyboard Accessibility) as some of the users tested for the interview might be asked to demonstrate using the website without a mouse (ex. using the coordinate navigation feature instead of choosing cells with a mouse). These nonfunctional requirements are related to test cases NFR-T18 and NFR-T19.

2. **Visual Clarity:** After entering a wrong answer, were you able to understand how the answer is wrong? (Yes / No / With Interviewer Help)

   **Rationale:** This question directly validates Functional Req 39 (Guided Learning: Cell Selection), as guided learning is built into the project since the stakeholders ask for a game that will teach octal and magic squares to its users. The user should be able to see what rule they are breaking when they are selecting a wrong cell, so they can learn where they are going wrong. It also validates Functional Req 41 (Guided Learning: Repeated Mistakes) as more mistakes should result in the

website getting more descriptive with its hints and essentially holding their hand to the answer. It should essentially act as how the interviewer will if the user got stuck (hence giving the option to choose "With Interviewer Help", which means that the user was unable to follow the website hints).

3. **Guided Learning Information Retention:** Are you able to complete the following octal magic sqaure after the tutorial? (Yes / No / After 3 mins)

   **Rationale:** After the guided learning section, the user is given a printed-out octal magic square (Sudoku-like). This is used to test out if the lessons and examples were taught well enough to see if the user can jump into the level system. The guided learning section has a test function of sorts where the website generates questions and verifies users' answers. This is also useful for the interviewer to infer how well the user will do in the actual game. This question validates that the premise of Usage Case 6 (Guided Learning and Tutorial) remains even if every step isn't followed.

4. **Reponse Time:** Does the screen refresh before you start thinking about the next step? (Yes / No)

   **Rationale:** This is an important question to ask as the current scoring system of the game is dependent on the time taken to solve the magic cube. If the user has to wait for their last answer to appear on the cube or for their answer to get verified, this can add minutes to the time spent playing, and greatly depress the user's final score. This question is directly related to Non-Functional Requirement - Test 1 (Cube Rotation Response Time) and Non-Functional Requirement - Test 2 (Cell-Value Editing Response Time), except it will be done by the users rather than found computationally. Users don't really care about the website's response times past a certain value as they don't have a small enough reaction time, and this question gives a reasonable response time to aim for in computational tests.

5. **Portability:** Are you able to access it via your favoured browser? (Yes / No)

   **Rationale:** This is an important question to ask as different browsers might have different nooks and crannies that might pose difficulties

(if the website has to allow usage on older browsers) or benefits (if a browser has an efficient implementation for displaying 3D models). Having a portable project also allows the team to create a browser-based mobile implementation as a stretch goal. This is related to Non-Functional Requirement - Test 9 (Browser Compatibility) and Non-Functional Requirement - Test 10 (Browser-Based Deployment). Although a tester (on the Capstone team) can easily try to go through all major browser implementations, letting the user do it gives the user a sense of comfort as they are using the browser they are used to.

6. **Account Creation:** After creating an account, do you see your other scores? (Yes / No)

   **Rationale:** While the team can easily simulate creating accounts and running games on multiple different accounts, they will still approach building the UI related to account creation as people that used multiple UIs. The useability might seem normal for software engineers, but hard to follow for younger students interested in learning octal, or older gentlemen not familiar with browser games (Persona 2). Navigating the account creation and using it to check scores will give the interviewers a good handle on how easy it is follow the game's flow. User Interviews will give a nice qualitative assessment to the quantitative assessments from Non-Functional Requirement - Test 15 (User Data Isolation) and Non-Functional Requirement - Test 16 (Authentication Secret Protection).

# 7 Appendix — Generative AI Use Disclosure

## 7.1 AI Tools Used

For sections 4.1 and 4.3, Claude Sonnet 5.5 (Medium) by Anthropic was used.

GPT-5.6 Luna was used in support of reviewing in section 4.2

Microsoft Copilot version 156.0 was used in support of researching and reviewing Section 3 of this document.

DeepSeek-V4.1-Flash was used for formatting and error checking in Sections 2 of the document.

## 7.2 How and Where Used

Claude was used to help draft the system tests for the functional requirements in Section 4.1. Using the functional requirements from Section 6.2 of the SRS as context, the system tests were created, with related requirements grouped into one test. Claude was also used to build the functional requirements traceability table in Section 4.3.

For Section 3, Generative AI was used in the context of receiving the project description and rubric as well as a draft of the document and suggesting additions and/or modifications. In this context, Generative AI was used as a reviewer for the document with the task of checking high-level concepts such as meeting all the rubric requirements and document structure/clarity. This document was originally written manually using original ideas and thinking. Once completed, the document, along with the guidelines for the document, was entered. This approach was generally used for the entire document, section by section, to collect feedback. One additional task that Generative AI was particularly useful for was ensuring that no points were missing from the various checklists in Section 3. After the original checklist was written, Generative AI would suggest other potential points (or modifications to existing points) that would enhance the checklist and make it more complete.

For section 4.2, generative AI was given my current rough draft and was asked to ensure that what was there was legible and suggest additions if deemed necessary.

For Section 2, Generative AI was given the written text and asked to return it without spelling errors.

## 7.3 Verification of AI-Generated Content

The system tests were checked against the functional requirements in the SRS and the traceability table against the tests. Scoring and octal calculations were recomputed, and one incorrect example was corrected before inclusion. The team remains responsible for the accuracy and completeness of all content in this document, regardless of whether it was generated by AI or written by a team member.

The suggestions provided by Microsoft Copilot for Section 3 were not implemented directly. Instead, to ensure the content was reasonable/quality and fit the tone and intent of our document, the comments were reviewed by a team member and then implemented manually. Generative AI was not used to create whole paragraphs or generate facts independently.

For Sections 2 and 4.2, all suggestions by generative AI were manually reviewed and updated accordingly before adding to the final document.

# 8 Appendix — Reflection

The information in this section will be used to evaluate the team members on the graduate attribute of Lifelong Learning.

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

1. What went well while writing this deliverable?

2. What pain points did you experience during this deliverable, and how did you resolve them?

3. What knowledge and skills will the team collectively need to acquire to successfully complete the verification and validation of your project? Examples of possible knowledge and skills include dynamic testing knowledge, static testing knowledge, specific tool usage, Valgrind etc. You should look to identify at least one item for each team member.

4. For each of the knowledge areas and skills identified in the previous question, what are at least two approaches to acquiring the knowledge or mastering the skill? Of the identified approaches, which will each team member pursue, and why did they make this choice?

## Team Answers

Questions 3 and 4 have been answered as a team.

3. One key skill that the team will need to acquire is learning to properly use the performance measurement tools: cProfile, Timeit, Tracemalloc,

and Mitata. Our team is not very familiar with these tools beyond their core purpose. However, the tools will be very useful in ensuring that our software meets performance standards. Jacob will focus on learning these tools as described in the answer to the next question.

Another main knowledge point that the team will need is knowledge of data encryption as well as the network inspection tools required to test the nonfunctional security requirements. This will allow us to be confident in our data security which is a critical point whenever handling any user data in an application.

Another skill we will need is automated testing of the browser interface, since many of the system tests in Section 4.1 are run by a script that clicks cells, enters answers, and reads the screen. Allen will focus on this skill.

Another skill we will need is testing of the asynchronous components of our system which will be tackled by Amos.

Another knowledge area that the team has to learn is applying functional languages to prove that cube states are valid. This can be important when generating new cube states (a stretch goal discussed with the TA) or ensuring that if the user chose the wrong cell, it can dynamically realize that the user chose the wrong cell, rather than it being baked into the code (hard written).

4. Jacob is focusing on learning the performance measurement tools: cProfile, Timeit, Tracemalloc, and Mitata. The first approach to mastering this skill is reading the documentation on the different libraries. Jacob will first focus on reading the quick start guides and introductions and then read method documentation as required. A second approach to mastering this skill is reviewing example uses of the libraries. The tools are very popular and as such, have been applied to many projects. By reviewing projects that are similar to this project, Jacob can gain a deeper understanding of how to properly utilize the tools.

   Allen is focusing on learning automated browser interface testing. The first approach is reading the documentation and getting started guides for a browser testing tool such as Playwright. The second approach is to practice by writing a few of the system tests from Section 4.1 (for example, ST-FR-1 and ST-FR-4) against an early version of the game, so the learning is applied directly to our own project.

Nehan will be learning and applying what he learned side by side with his functional programming course. This gives him access to a professor and TA that he can ask for help if he needs it, alongside countless online resources. As this isn't important for the first revision of the app or PoC, Nehan can analyze if he is able to take on this task (a stretch goal verifying generated cubes). If not, he is always able to use an object-oriented language to validate game states.

Josh is looking into network inspection tools to attempt to further his knowledge of these tools. The first approach to be made is doing wide reaching research to find a tool suited to our project to ensure we get the best fit. Then to truly deepen their knowledge they can also read further into the documenetation and use of the chosen tool.

Amos will focus on learning how to handle testing asynchronous libraries such as asyncio. The first aproach to mastering this is reading available documentation on asyncio and pytest-asyncio and expaning this to any asynchronous libraries we finalize our system with. The next aproach will be reviewing the guides to asynchronous testing online and referencing this with video guides related to the skill.

## Josh Engel

1. I think most of the work I completed for 4.2 went quite well. Being that I had worked on the non-functional requirements for the SRS document I already had a solid grasp on how I was hoping to test them and so just had to put it to words. This also allowed for me to easily fit the tests to the earlier defined fit critera for the non-functional requirements.

2. One pain point I found during this was for NFR-17. It was a struggle to try and figure out how to make this requirement testable being that it is more just a attribute of the code. In the end I decided on just making it a code inspection but my mind was so focused on tests it took me a while.

## Jacob Brodersen

1. Laying out the roles and responsibilities for each team member in the Verification and Validation Team table went well. This went well because we had already laid out the base responsibilities in the Team

Member Roles section of the Development Plan. From this, I simply needed to extend those roles/responsibilities to verification and validation. However, I have designed the VnV roles/responsibilities so that the same individual doing the VnV section does not have the same roles/responsibilities that they had for the design/implementation. This works best because the people who worked on a given component should not be the one reviewing it as to avoid compounding a single perspective on a section of work.

2. One pain point I experienced during this deliverable was determining what strategies to use for Design Verification. To resolve this, I spent time reviewing the example documents as well as the prompt comments for the section. This review, along with some brainstorming, allowed me to determine strategies that would be applicable and effective at verifying the design for this project.

## Amos Edeaghe

1. While working on the general information for this document, I found that finding the information was very simple since we had covered a lot of it already while working on other documents or speaking with Dr. Rapoport.

2. During this deliverable, I was unable to attend our weekly TA meeting and had a few unresolved questions on the nature of the work that needed to be done; thankfully, I was able to discuss with my team members to refine my ideas and clear up confusion.

## Allen Tong

1. What went well for me was writing Section 4.1. Since I wrote the functional requirements in the SRS, I already knew how each requirement was supposed to behave, so the tests came naturally from them. I could work out the expected values directly from the rules, such as the scoring formula and the octal subtraction constants, and group related requirements into a single test. It also made the traceability table in Section 4.3 straightforward because I knew which requirements each test needed to cover.

2. One pain point was the number of tests. My first draft had a test for almost every requirement, which came to 46 tests and was too much for this document. I resolved this by grouping related requirements together, which brought it down to 22 tests while still covering all 55 requirements.

## Nehan Munir

1. This was the most effortless deliverable I have done so far, and while the tasks (Interview Questions) I have been assigned were simple, also having a clearly defined mental visualization of how the final project will look also helps. This mental image has been created after having focused and etensive deliverables that outline the flow of the game, alongside multiple in-person and virtual meetings with the team and the supervisor.

2. One of the pain points that I had was trying to link the SRS and VnV test plans with the user useability questions. I wanted to ensure that it contains the bidirectionality and linkage so that when looking back at this document while writing our code, I can see how we arrived at each decision and what else it is related to. I solved this problem by realizing what I was asked to do, which doesn't really correspond with other elements of the VnV plan, which was mostly about test the team can do indiidually (either computationally simulating workload or using static analysis tools), while my parts had to be done with a tech illiterate user in mind. This led me to leaning in to using my questions as a qualitative assessment substitute to the quantitative tests in other sections.